ADR-0003 : Backend cryptographique pur Rust, HS256 uniquement

- **Statut :** Accepté
- **Date :** 2026-07-21
- **Décideurs :** Driss OUAAZIZ — Software Architect

Contexte et problème

La signature et la vérification des jetons JWT reposent sur la bibliothèque `jsonwebtoken`. Sa version 10 introduit un **choix explicite du fournisseur cryptographique** via des *features* : soit `aws_lc_rs` (adossé à la bibliothèque C `aws-lc-rs`), soit `rust_crypto` (implémentation entièrement en Rust). Aucune n'est activée par défaut ; en leur absence, l'exécution échoue au démarrage (« impossible de déterminer le fournisseur cryptographique »).

Par ailleurs, l'objectif de production est de livrer un **binaire entièrement statique** sur base distroless minimale (voir ADR-0004). Toute dépendance à une bibliothèque C (compilation `cmake`/toolchain, édition de liens dynamique) va à l'encontre de cet objectif et réintroduit des composants système susceptibles de porter des vulnérabilités (OpenSSL notamment).

FileFlow n'émet et ne vérifie **que des jetons HS256** (HMAC-SHA-256, clé symétrique) ; aucun usage de signatures asymétriques (RS256, ES256…) n'est prévu.

Facteurs de décision

- Compatibilité avec une compilation statique musl sans dépendance C.
- Éviter OpenSSL et, plus largement, réduire la surface de dépendances système.
- Couvrir strictement le besoin réel (HS256) sans embarquer de code inutile.
- Reproductibilité et simplicité de la chaîne de build.

Options envisagées

1. **`aws_lc_rs` (défaut usuel)** — Performant, mais requiert une toolchain C (`cmake`, compilateur) au build et produit une dépendance native ; complexifie et fragilise la compilation statique musl.
2. **`rust_crypto`** *(retenu)* — Implémentation pure Rust (HMAC, SHA-2, etc.), sans dépendance C, compatible sans friction avec la cible musl statique.

Décision

Déclarer la dépendance ainsi :

```
        jsonwebtoken = { version = "10", default-features = false, features =
["rust_crypto"] }
```

- `default-features = false` évite d'embarquer `use_pem` et les dépendances associées, inutiles puisque seule la clé symétrique HS256 est employée.
- `rust_crypto` fournit un fournisseur cryptographique **pur Rust**, condition nécessaire au binaire statique musl et à l'absence d'OpenSSL dans l'image.

Conséquences

Positives

- Aucune dépendance cryptographique C ; build statique musl sans toolchain native supplémentaire.
- OpenSSL absent de l'image de production, ce qui supprime toute une classe de CVE système (voir ADR-0004).
- Périmètre cryptographique aligné sur le besoin réel (HS256).

Négatives

- La *feature* `rust_crypto` est monolithique : elle tire l'ensemble des primitives RustCrypto, dont le crate `rsa` (support RS256), **non utilisé** par FileFlow.

Risques résiduels

- Le crate transitif `rsa` fait l'objet de l'avis **RUSTSEC-2023-0071** (attaque « Marvin », canal temporel sur RSA), **sans correctif disponible**. Ce chemin est **inatteignable** : FileFlow n'exécute jamais d'opération RSA (HS256 exclusivement). L'avis est donc **accepté et documenté** dans `backend/.cargo/audit.toml` (ignore justifié), avec consigne de retrait de l'ignore si des JWT signés RSA venaient à être introduits.
- Un passage futur à des signatures asymétriques imposerait de revoir ce choix (activation d'une autre *feature* ou d'un autre fournisseur).

Références

- Avis RUSTSEC-2023-0071 : https://rustsec.org/advisories/RUSTSEC-2023-0071.
- Code : `backend/Cargo.toml`, `backend/.cargo/audit.toml`, `backend/src/auth.rs`.
- ADR-0002 (révocation JWT) et ADR-0004 (image statique musl).